

DỰ ÁN: DULICHAI - TRỢ LÝ DU LỊCH THÔNG MINH TOÀN DIỆN

Báo cáo Tuần 3: Quy trình Kỹ thuật & Luồng Bản đồ

Xây dựng Lõi Kỹ thuật (Engineering Core) định hướng Offline-First.

Nhóm 2: Hữu Nghĩa, Trường Thịnh, Anh Khoa | GVHD: Trần Thế Vũ



Sản phẩm Bàn giao (What)



Hệ thống Google Maps hoàn thiện luồng định vị và xin quyền vị trí.



Hệ thống SOS tích hợp dữ liệu offline, truy cập khi mất mạng.



Kết nối màn hình Planner với Room Database.



Chuẩn hóa Weather API adapter.

Trọng tâm Kỹ thuật: Triết lý Engineering Core

Giai đoạn thiết lập nền tảng bắt buộc. Đội ngũ tập trung chứng minh năng lực thực thi thông qua quy trình làm việc Git chuyên nghiệp, duy trì mã nguồn ổn định tuyệt đối trước khi tích hợp các SDK AI phức tạp (Gemini) vào hệ thống.

Chiến lược Phân nhánh (Git Branching Architecture)

Tách biệt môi trường phát triển giúp duy trì mã nguồn sạch và giảm thiểu rủi ro xung đột khi tích hợp SDK.

feature/*

Phát triển tính năng riêng lẻ
(VD: feature/offline-roomdb, feature/maps).

develop

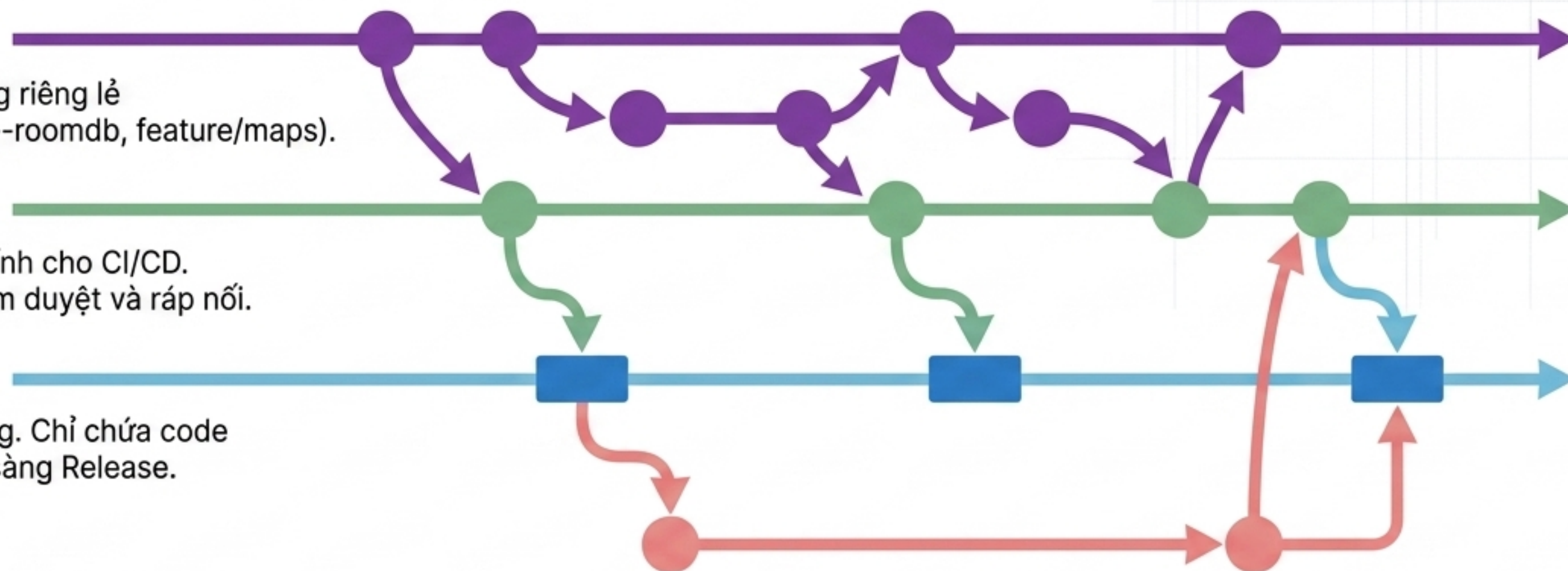
Nhánh tích hợp chính cho CI/CD.
Nơi code được kiểm duyệt và ráp nối.

main

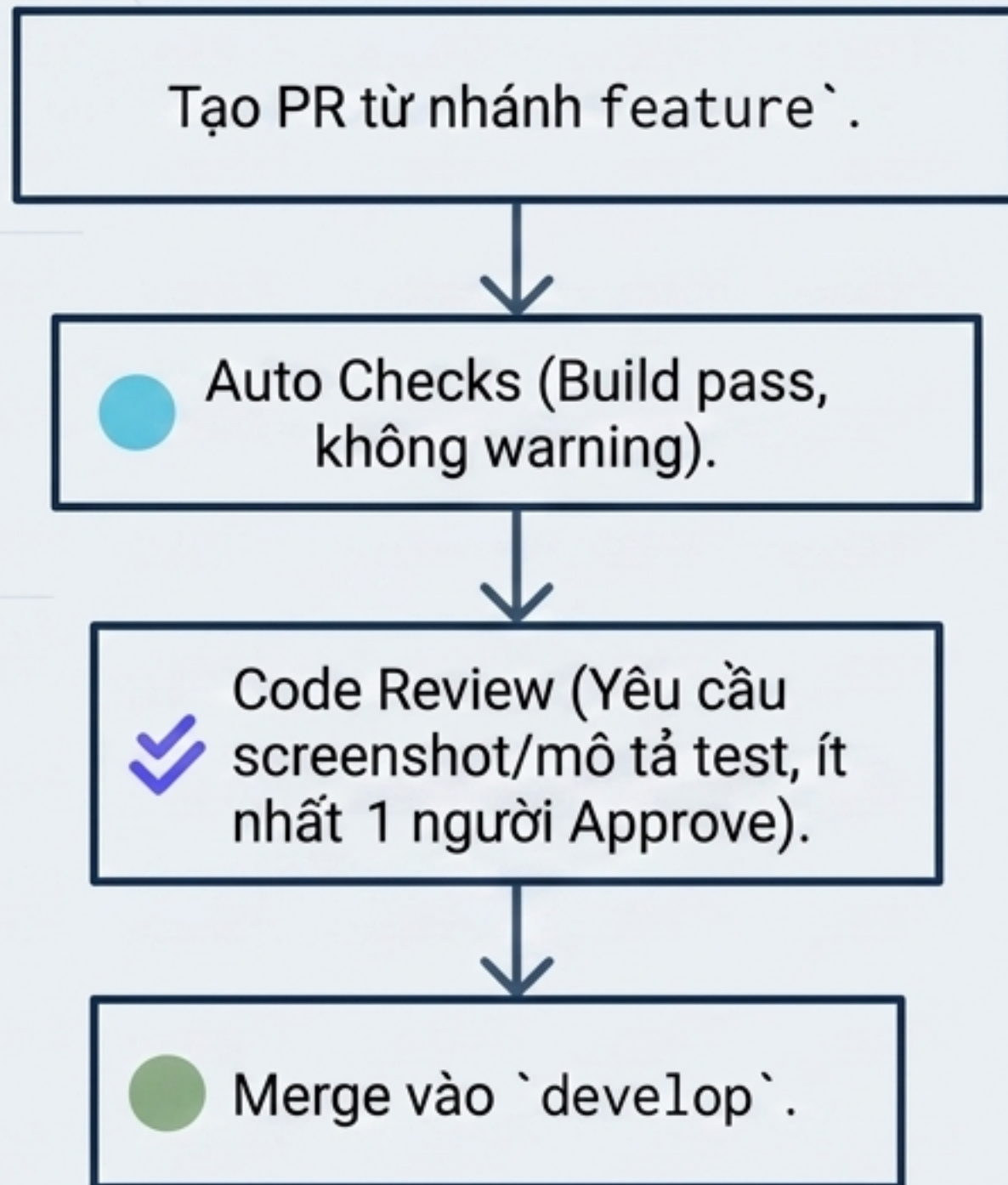
Điểm đến cuối cùng. Chỉ chứa code
ổn định nhất, sẵn sàng Release.

hotfix/*

Nhánh rẽ khẩn cấp từ main để vá lỗi trực tiếp.



Luồng Kiểm duyệt PR



Chuẩn Conventional Commits

Cấu trúc: type(scope): description
(feat, fix, refactor, docs, chore)

✓	<code>feat(maps): add location permission flow</code> <code>fix(db): resolve race condition in Room DAO</code>
✗	<code>update code</code> <code>fix bug</code>

Nguyên tắc “Phòng hơn Chữa” (Xử lý Xung đột)

Frequent Merges

Kéo (Pull) dữ liệu từ nhánh develop hàng ngày để giảm độ trễ code.

Small PRs

Giữ Pull Request với phạm vi nhỏ, giúp review nhanh và cô lập xung đột.

Sync Before PR

Bắt buộc rebase/merge nhánh develop trước khi mở PR.

No Force Push

Khóa hoàn toàn lệnh force push trên các nhánh làm việc chung.

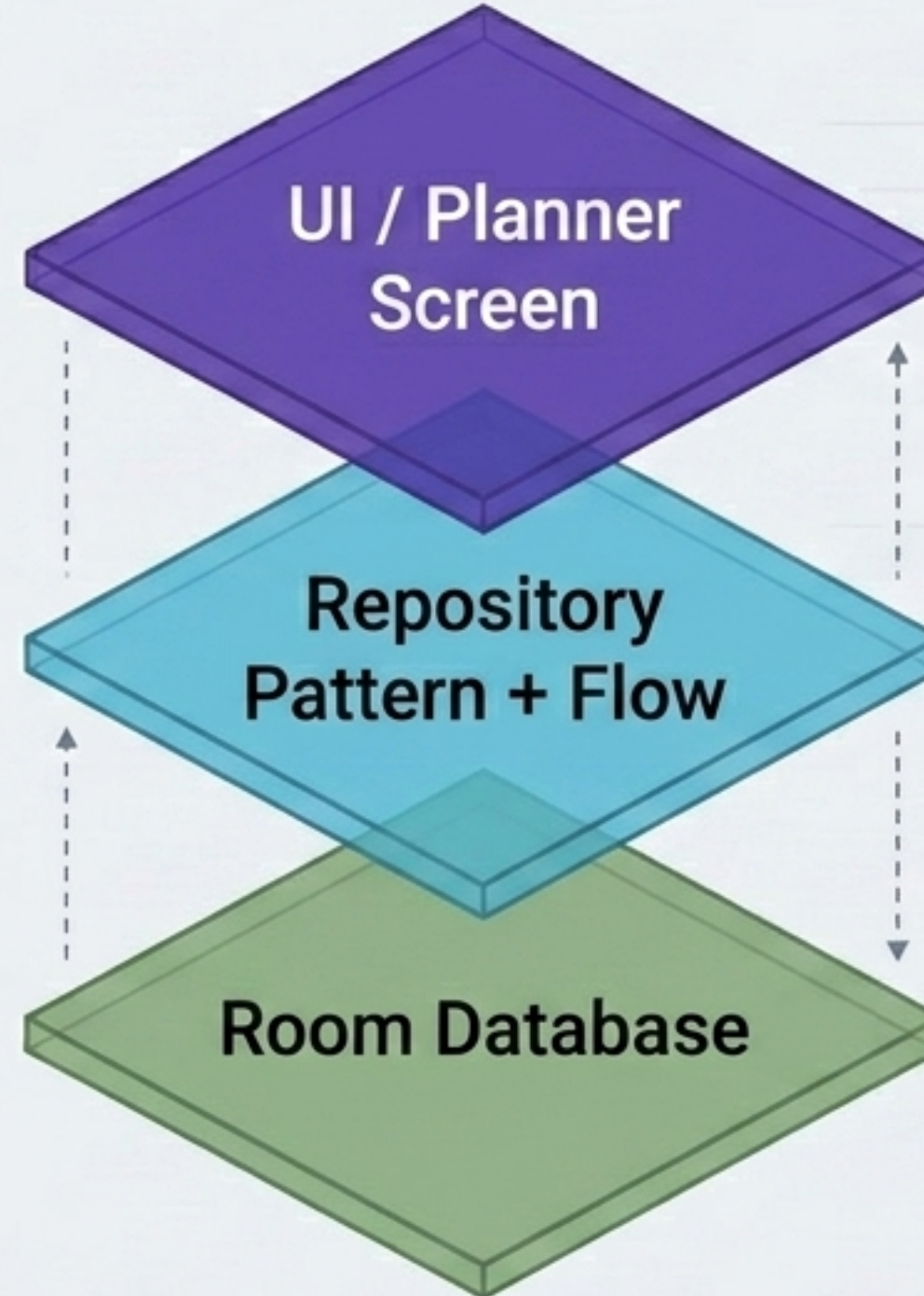


Tech Deep Dive: Kiến trúc Offline-First

Tiến độ: Hoàn thành 90%



Mục tiêu: Dữ liệu lịch trình hiển thị tức thì khi không có mạng.



Tiêu thụ dữ liệu từ ViewModel

Bọc Room DAO, xử lý bất đồng bộ, chống race condition

Room Database Schema: Trip, ItineraryItem, Document

Tech Deep Dive: Google Maps SDK

Location Services

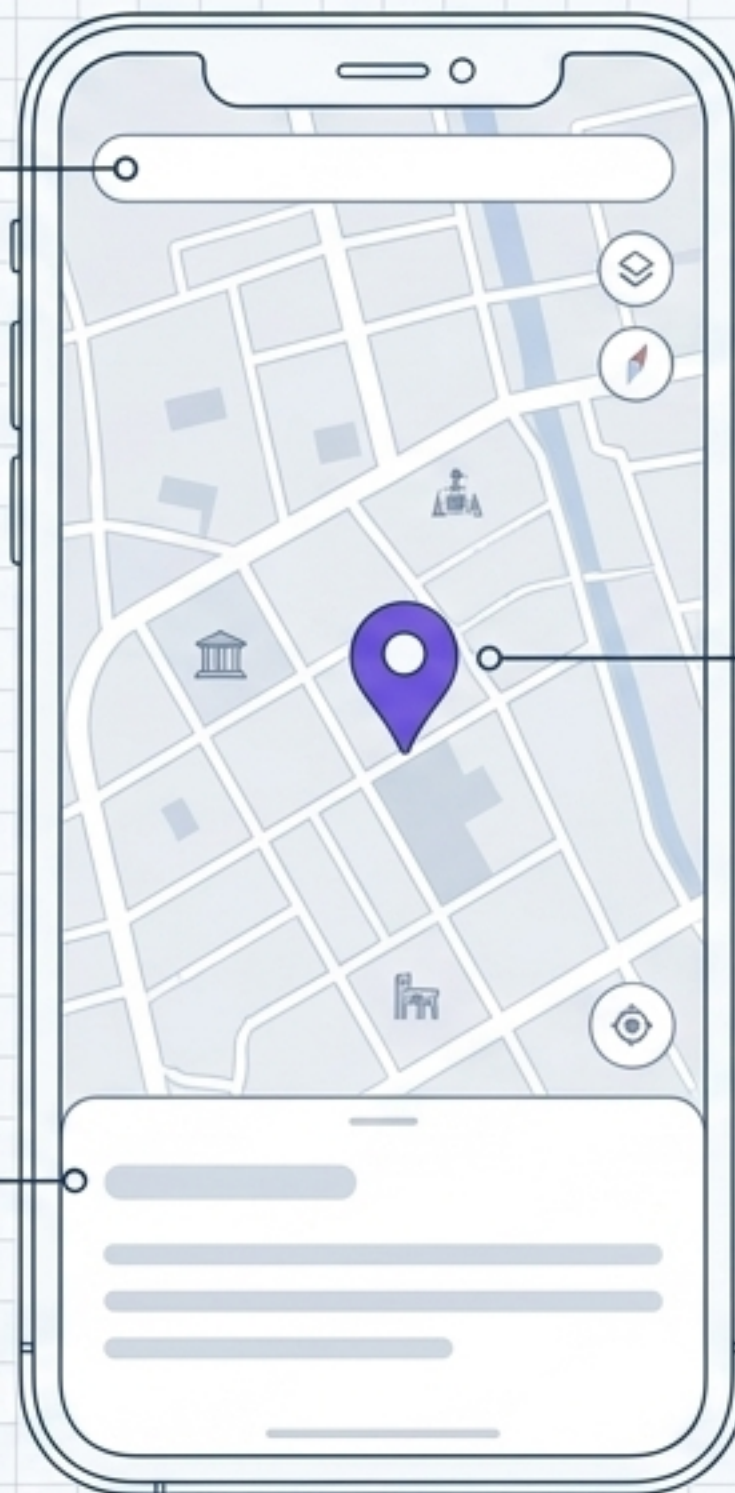
Xin quyền truy cập và lấy tọa độ người dùng theo thời gian thực (Location Permission Flow).

UI/UX

Tích hợp Bottom Sheet component để hiển thị thông tin địa điểm chi tiết mà không rời khỏi bản đồ.

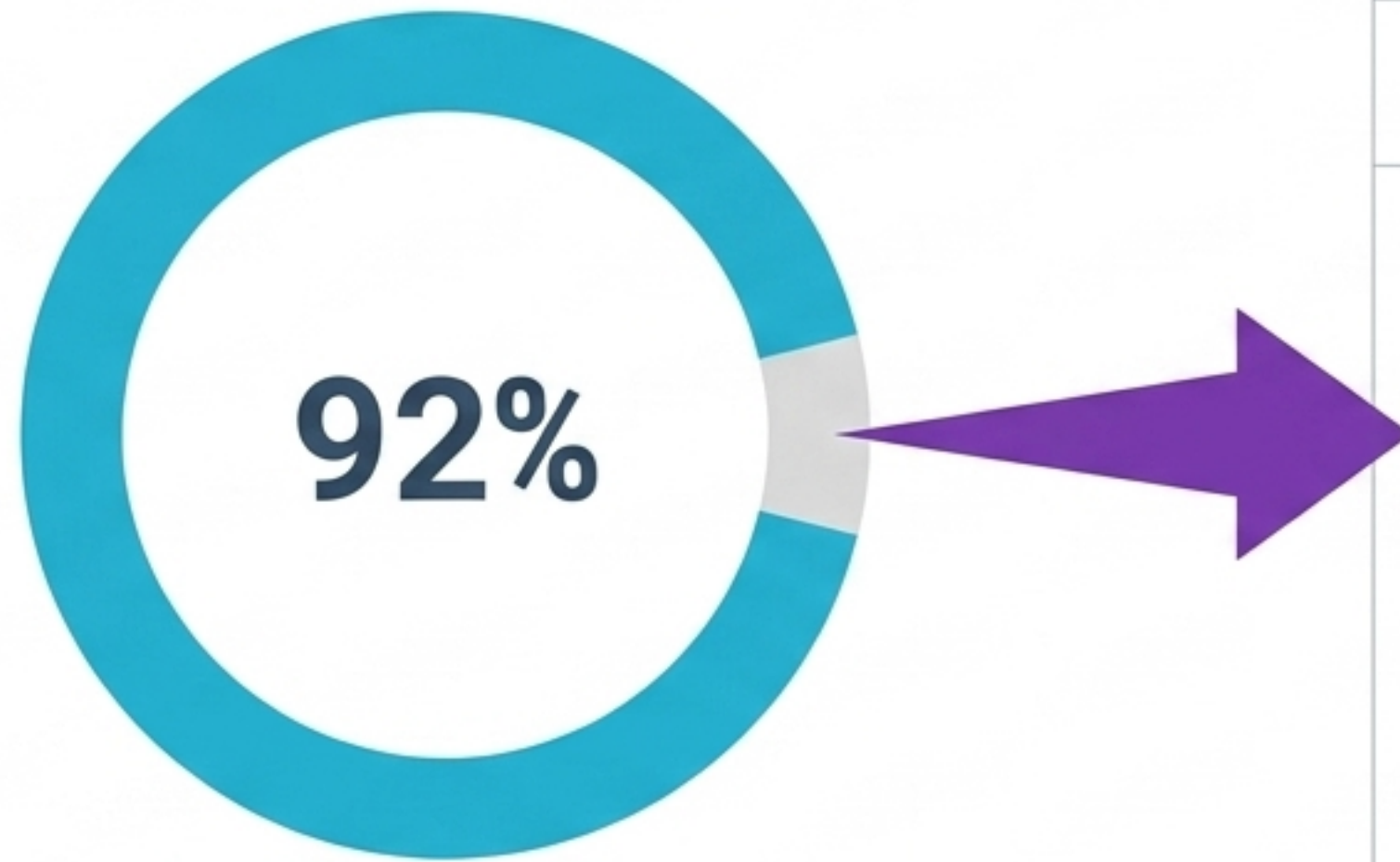
Marker Custom

Xử lý hiển thị các Landmark (điểm đến) trực tiếp trên bản đồ.



Tổng hợp Nguồn lực & Thời gian

Resource Gauge



Kế hoạch: 18h
Thực tế: 16.5h
Khoảng hụt (Gap): 1.5h

Xoay trục Kỹ thuật (Pivot)

- **Nguyên nhân:** Hiệu suất tăng nhờ ứng dụng AI tools trong viết code cơ bản.
- **Hành động:** Dồn 1.5h thời gian dư này vào việc viết Unit Test cho module Maps để đảm bảo độ bao phủ lỗi. Tạm thời cắt giảm trau chuốt UI chưa cần thiết.

Ma trận Chẩn đoán Rủi ro

Kiến trúc Rủi ro (Risk)	Mức độ Tác động (Impact)	Giao thức Xử lý (Mitigation Protocol)
Xung đột build Gradle khi tích hợp Maps + Gemini AI.	Mức CAO	Sử dụng Version Catalog (<code>libs.versions.toml</code>) để quản lý tập trung; phân công 1 người quản lý dependency.
Race condition trong Room DB khi gọi Coroutines.	TRUNG BÌNH	Sử dụng Kotlin Flow + ViewModel để quan sát bất đồng bộ; viết Unit test cho mọi method DAO.
Áp lực 5 tuần gây tích lũy Nợ Kỹ thuật (Tech Debt).	RẤT CAO	Kỷ luật ưu tiên luồng dữ liệu cốt lõi (MVP), sẵn sàng cắt giảm tính năng phụ (UI polish) nếu trễ tiến độ.

Tầm nhìn & Bước tiếp theo (The Horizon)

Trọng tâm Tuần 4

- Tích hợp CameraX (Xử lý hình ảnh từ camera).
- Triển khai Gemini AI Hybrid (Kết hợp xử lý Cloud + Local).
- Hoàn thiện luồng dịch thuật tự động.



Nhóm 2 - Dự án DuLichAI

github.com/truongthinh994/DuLichAI

**Lỗi kỹ thuật đã sẵn sàng.
Tuần 4: Kích hoạt Trí tuệ Nhân tạo.**